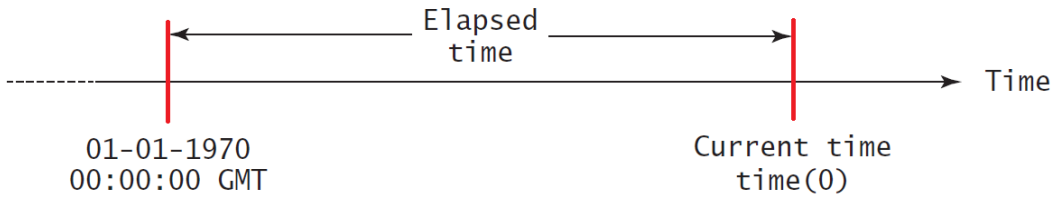
The time(0) Function in C++

The time(0) function, in the ctime header file, returns the current time in seconds elapsed since the time 00:00:00 on January 1, 1970 Greenwich Mean Time (GMT).



You can use the function time(0) to determine the current time on your place as demonstrated by the following code;

Ex_0601.cpp

```cpp
#include <iostream>
#include <ctime>
using namespace std;

int main()
{
    // Obtain the total seconds since the midnight, Jan 1, 1970
    int totalSeconds = time(0);
    int offset;
    cout << "Enter the timezone offset to Greenwich Mean Time (GMT): ";
    cin >> offset;

    int currentHour = (totalSeconds / (60*60) + offset) % 24 ;
    int currentMinute = (totalSeconds / 60) % 60;
    int currentSecond = totalSeconds % 60;

    // Display  results
    if (currentHour > 12)
        cout << "The current time is " << currentHour - 12 << ":"
             << currentMinute << ":" << currentSecond << " PM" << endl;
    else
        cout << "The current time is " << currentHour << ":"
             << currentMinute << ":" << currentSecond << " AM" << endl;

    return 0;
}
```

Generating Random Numbers

To generate a random number, use the rand() function in the cstdlib header file. This function returns a random integer between 0 and RAND_MAX. RAND_MAX is platform-dependent constant. My computer gives RAND_MAX = 32767

Let us use the following code to generate 10 random numbers in the rang 0:999

Ex_0602.cpp

```
#include<iostream>
using namespace std;

 int main()
{
    int i;
    cout << "ten random numbers for the range 0 to 999\n" << endl;
    for (i = 0 ; i < 10 ; i++)
       cout << rand() % 1000 << ", ";
    return 0;
}
```

When I run this code it gives me the following output;

```
ten random numbers for the range 0 to 999

41, 467, 334, 500, 169, 724, 478, 358, 962, 464,
```

When I run it for the second time it gives me the following output;

```
ten random numbers for the range 0 to 999

41, 467, 334, 500, 169, 724, 478, 358, 962, 464,
```

When I run it for the third time it gives me the following output;

```
ten random numbers for the range 0 to 999

41, 467, 334, 500, 169, 724, 478, 358, 962, 464,
```

In fact, the above code will always give the same random numbers. **Why?**

The reason is that rand() function's algorithm uses a value called the *seed* to control how to generate the numbers. By default, the seed value is 1. If you change the seed to a different value, the sequence of random numbers will be different. To change the *seed*, use the srand(seed) function in the cstdlib header file. To ensure that the *seed* value is different each time you run the program, use time(0).

Let us modify the above code by introducing the time as a seed value for random algorithm as follows;

Ex_0603.cpp

```cpp
#include<iostream>
#include<ctime>
using namespace std;

 int main()
{
    srand(time(0));
    int i;
    cout << "ten random numbers for the range 0 to 999\n" << endl;
    for (i = 0 ; i < 10 ; i++)
       cout << rand() % 1000 << ", ";

    return 0;
}
```

When I run this code it gives me the following output;

```
ten random numbers for the range 0 to 999

527, 181, 589, 977, 344, 784, 899, 455, 905, 288,
```

When I run it for the second time it gives me the following output;

```
ten random numbers for the range 0 to 999

66, 196, 57, 495, 313, 103, 166, 974, 573, 664,
```

When I run it for the third time it gives me the following output;

```
ten random numbers for the range 0 to 999

160, 219, 830, 436, 524, 906, 123, 387, 889, 210,
```

The problem has been solved. Different random numbers are obtained from each.

Simple File Input and Output

You used the cin to read input from the keyboard and the cout to write output to the console. You can also read/write data from/to a file. This section introduces simple file input and output.

Writing to a File

The following program shows how to open an output file and write the data to this file and how to close that file.

Ex_0604.cpp

```cpp
#include <iostream>
#include <fstream>  // For file streams
#include<iomanip>
using namespace std;

int main() {
    // Create and open a file for writing
    ofstream outputFile("my_data.txt");

    // Check if the file was opened successfully
    if (outputFile.is_open()) {
        // Write data to the file
        outputFile << "Hello, C++ file writing!" << endl;
        outputFile << "This is the second line." << endl;
        outputFile << "The answer is: " << 42 << endl;
        // Close the file
        outputFile.close();
        cout << "Data successfully written to my_data.txt" << endl;

    // display an error message using standard error stream cerr
    } else {
        cerr << "Error: Unable to open the file for writing." << endl;
    }

    return 0;
}
```

Note that: cerr is a standard error stream object used for displaying error messages.

Reading from a File

The following program shows how to open an input file and read the data to this file and how to close that file.

Ex_0605.cpp

```cpp
#include <fstream>
#include <iostream>
#include <string> // For std::string and getline
using namespace std;

int main() {
    ifstream inFile("example.txt"); // Open the file

    if (inFile.is_open()) { // Check if file opened successfully
        string line;
        while (getline(inFile, line))
        { // Read line by line
            cout << line << endl;
        }
        inFile.close(); // Close the file
    } else {
        cerr << "Error opening file for reading." << endl;
    }
    return 0;
}
```

Repeating Structures (Loops)

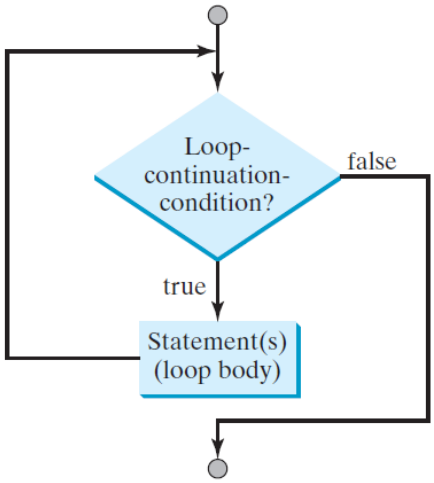
Repeating structures or Loops in C++ are control flow structures that enable the repeated execution of a block of code based on a specified condition. They are fundamental for automating repetitive tasks and iterating over collections of data. C++ provides three types of loop statements: while loops, do-while loops, and for loops. The structure of these types will be introduced here.

The while Loop

A while loop is a *condition-controlled loop* that repeatedly executes a block of code as long as a specified condition remains true. It is particularly useful when the number of iterations is not known beforehand and depends on a dynamic condition.

The syntax for the while loop is

```
while (loop-continuation-condition)
{
  // Loop body
  Statement(s);
}
```



The following examples demonstrate the while loop.

Example

Write a C++ program to print numbers from 1 to n using while-loop.

Ex_0606.cpp

```cpp
#include <iostream>
using namespace std;
int main()
{
    int n, i = 1;
    cout << "Enter number: ";
    cin >> n;
    cout << endl;
    while (i <= n)
    {
        cout << i << ", ";
        i++;
    }
    return 0;
}
```

When you run this program, you obtain the following results;

```
Enter number: 20

1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
```

Example

Write a C++ program to calculate the factorial of n.

Ex_0607.cpp

```cpp
#include <iostream>
using namespace std;

int main()
{
    int a = 1, num1, factorial = 1;

    cout << " Input a number to find the factorial: ";
    cin >> num1;

    while ( a <= num1 )
    {
        factorial = factorial * a;
        a++;
    }

    cout << " The factorial of " << num1 << " = " << factorial << endl;
    return 0;
}
```

Example

Write a C++ program to calculate the sum of the following series;

$$\sum_{i=1}^{n}(i)^{i} = 1^1 + 2^2 + 3^3 + \ . \ . \ . \ + n^n$$

Ex_0608.cpp

```
#include<iostream>
#include<cmath>
using namespace std;

int main()
{
    int i=1, n, sum = 0;

    cout << " Input the value for nth term: ";
    cin >> n;

    while (i <= n)
    {
        sum += pow(i,i);
        cout << i << "^" << i << " = " << pow(i,i) << endl;
        ++i;
    }

    cout << " The sum of the above series is: " << sum << endl;

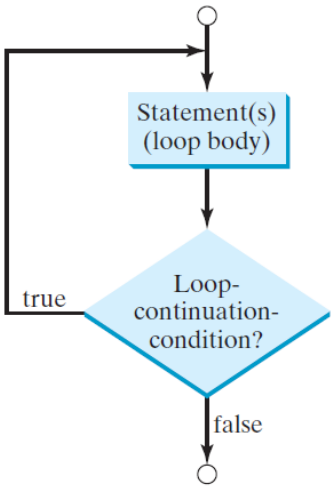
    return 0;
}
```

The do-while Loop

A do-while loop is the same as a while loop except that it executes the loop body first and then checks the loop continuation condition.

The do-while loop is a variation of the while loop. Its syntax is as follows:

```
do
{
  // Loop body;
  Statement(s);
} while (loop-continuation-condition);
```



The following example shows how to use do-while loop.

Write a C++ program to enter a sequence of integer numbers, then calculate and display the summation of these numbers. The calculation terminates when you input an integer of zero value.

Ex_0609.cpp

```cpp
#include <iostream>
using namespace std;

int main()
{
    // Keep reading data until the input is 0
    int data = 0, sum = 0;

    do
    {
    sum += data;

    // Read the next data
    cout << "Enter an integer (the input ends if it is 0): ";
    cin >> data;
    }
    while (data != 0);

    cout << "The sum is " << sum << endl;

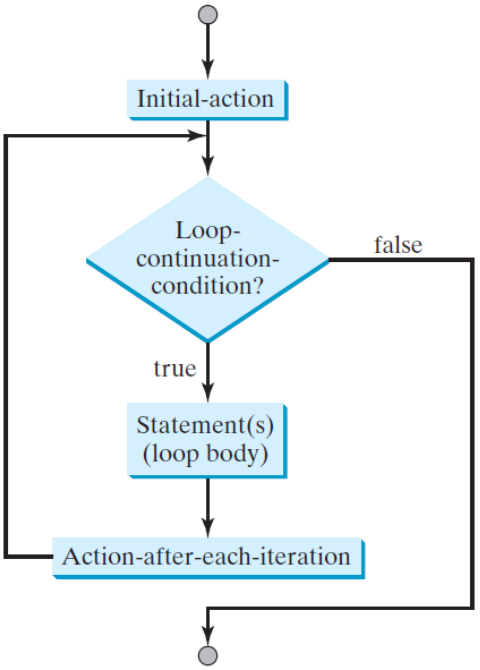
    return 0;
}
```

The for Loop

In C++, for loop is a *count-controlled loop* that is used to execute a block of code repeatedly for the given number of times. It is generally preferred over while and do-while loops in case the number of iterations is known beforehand.

In general, the syntax of a for loop is as shown below:

```
for (initial-action; loop-continuation-condition;
     action-after-each-iteration)
{
  // Loop body;
  Statement(s);
}
```



Example

Write a C++ program to enter to find the sum of the first 10 natural numbers.

Ex_0610.cpp

```cpp
#include <iostream>
using namespace std;

int main()
{
    int i, sum = 0;

    for (i = 1; i <= 10; i++)
    {
        cout << i << " ";
        sum = sum + i;
    }
    cout << "\n The sum of first 10 natural numbers: " << sum << endl;
}
```

When you run this program, you obtain the following results;

```
1 2 3 4 5 6 7 8 9 10
 The sum of first 10 natural numbers: 55
```

Example

Write a C++ program to calculate the factorial of an integer n.

Ex_0611.cpp

```cpp
#include <iostream>
using namespace std;

int main()
{
    int num1, factorial = 1;

    cout << " Input a number to find the factorial: ";
    cin >> num1;

    for (int a = 1; a <= num1; a++)
    {
        factorial = factorial * a;
    }

    cout << " The factorial of " << num1 << " = " << factorial << endl;
    return 0;
}
```

When you run this program, you obtain the following results;

```
Input a number to find the factorial: 10
The factorial of the given number is: 3628800
```

Example

Write a C++ program that prompts the user to enter an integer, determines whether the entered number is a prime number, and then displays an appropriate message indicating the result.

Ex_0612.cpp

```cpp
#include <iostream>
using namespace std;

int main()
{
    int number;
    bool isPrime = true;

    cout << "Enter an integer: ";
    cin >> number;

    if (number <= 1)
    {
        isPrime = false;
    }
    else
    {
        for (int i = 2; i <= number / 2; i++)
        {
            if (number % i == 0)
            {
                isPrime = false;
                break;
            }
        }
    }

    if (isPrime)
        cout << number << " is a prime number." << endl;
    else
        cout << number << " is not a prime number." << endl;

    return 0;
}
```